

3.3 The Transformer Block

Positions, multi-head attention, residuals, layer norm, and a token-wise MLP.

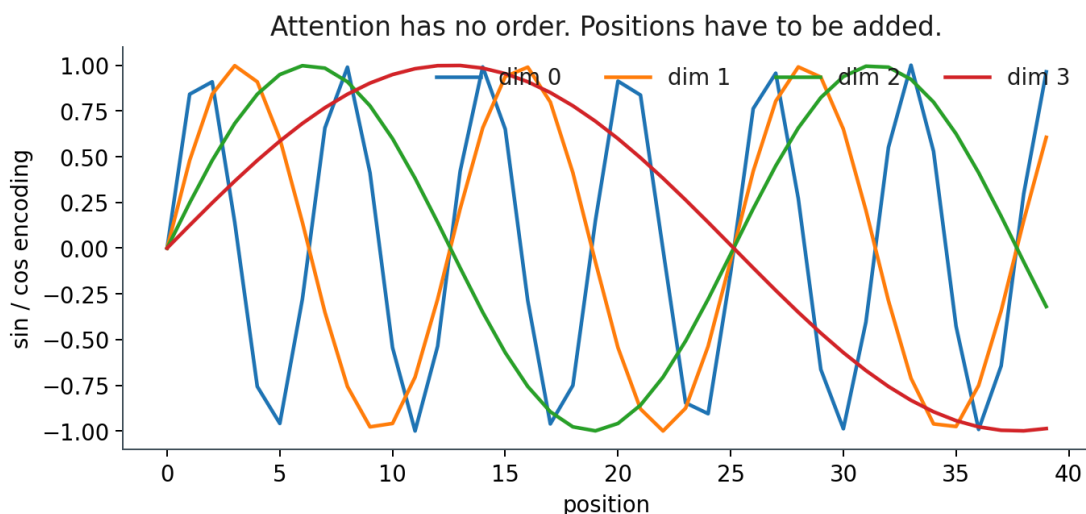
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A transformer **block** is attention, a residual add, layer norm, then a token-wise MLP, another residual and norm. Stack N of those. Attention mixes across positions; the MLP mixes across channels at one position.

1. Positions have to be added

Attention is a set operation: permute the tokens, the weights permute with them. Language is ordered.

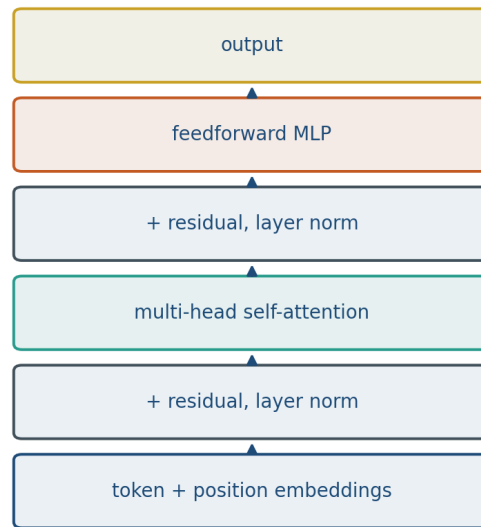
Positional encodings (sinusoids in the original paper, or learned vectors) are added to token embeddings before the first block.



Relative or rotary positions (RoPE) appear in later LLMs. The idea is the same: put order into the vectors because the mixing layer will not.

2. The block

One transformer block



Residual: add the input of the sublayer to its output. That is another highway, like the LSTM cell, so gradients can skip a messy attention map.

Layer norm: normalize across features of one token. Pre-norm versus post-norm is an implementation detail you will meet in Hugging Face configs.

MLP: usually two linear maps with a GELU, hidden width $4d$. Most of the parameters live here, not in attention.

3. Encoder–decoder

The original transformer has an **encoder** (bidirectional self-attention on the source) and a **decoder** (causal self-attention plus **cross-attention** into the encoder). Translation is the picture. GPT keeps only a decoder. BERT keeps only an encoder. You do not need both unless the task is sequence-to-sequence.

4. Practice

1. If you drop positional encodings, what two sentences become indistinguishable?
2. Residual connections help training. Which earlier idea in Week 2 is the cousin of that?
3. Where do most of the weights sit: W_Q, W_K, W_V or the MLP?